COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	10/15/26	**DATE SUBMITTED:**	10/20/26
NAME:	Dimaguiba, Lorenzo James M. Urbano, Vince Andrew F.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 13.1 Parallel and Multithreading

1. PROCEDURE OUTPUT

Screenshot

HOA13.1 > main.cpp > main()

```cpp
#include <iostream>
#include <string>
#include <thread>

void print(int n, const std::string &str){
    std::cout<<"Printing Integer: " << n << std::endl;
    std::cout<<"Printing String: " << str << std::endl;
}

int main(){
    std::thread t1(print, 10, "T.I.P");
    t1.join();
    return 0;
}
```

Output:

```
[Running] cd "c:\Users\TIPQC\Documents\VSCODE\HOA13
"c:\Users\TIPQC\Documents\VSCODE\HOA13.1\"main
Printing Integer: 10
Printing String: T.I.P
```

Analysis

The function works as intended. There isn't much difference compared to just calling the function into main; returning the same output anyway. However, how threads work differently is HOW they use CPU resources and WHEN they return an output or result. In this case, thread in this line of code is meant to pause or halt operations till that thread that was joined has finished execution.

Screenshot

```cpp
#include <iostream>
#include <vector>
#include <thread>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}
int main() {
    std::vector<std::string> s = {
    "T.I.P.",
    "Competent",
    "Computer",
    "Engineers"
    };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }
    for (auto &th : threads) {
        th.join();
    }
    return 0;
}
```

Output:

```
PS C:\Users\TIPQC\Documents\10-15-26>  ${env:DEBUGINFOD_URLS}='
exe' '--stdin=Microsoft-MIEngine-In-gna5idmt.xmr' '--stdout=Mic
ze11hq.tgj' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpre
0 : T.I.P.2 : Computer3 : Engineers1 : Competent
```

```
                                                    ${env:DE
exe' '--stdin=Microsoft-MIEngine-In-iwnr40ab.35
uhkwtr.4o3' '--dbgExe=C:\msys64\ucrt64\bin\gdb.
0 : T.I.P.2 : Computer

3 : Engineers1 : Competent

PS C:\Users\TIPQC\Documents\10-15-26>
```

Analysis

In these lines of codes, we expected that the output of the program would be according to their orders like from lines zero to one but instead the outputs are different each time they are executed. This can be due to the difference between computers' hardware. Moving onto the codes, std::vector is implemented to add values inside an array without a fixed number of values

	that can be allowed to the array. The function of threads.push_back() is to manage multiple execution of threads by storing the values inside the std::vector. The following "threads" for-loop joins all assigned threads that practically wait for each other to finish executing, once again pausing the main function, till, once finished, prints either or all at the same time.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Questions:

1. **Write a definition of multithreading and its advantages/disadvantages.**

   Dimaguiba:

   - Multithreading is often defined as a technique or process that divides executables into different units called "threads" allowing the program to run multiple or many processes simultaneously (most common being parallelism) or at the same time. Its main advantage is running multiple instances at the same time, effective CPU resource use (assigning how many threads to do a task), and shared memory access. Some disadvantages that come to mind is that it isn't really suitable for smaller-scale functions as it uses more resources than normal. Not all processes can run simultaneously and can prove unpredictable in some scenarios. Though it is more efficient, it significantly uses more CPU resources. It may also cause unforeseen results due to it pausing all operations, giving priority to the threads.

   Urbano:

   - Multithreading refers to the multiple execution of more than one thread of instructions. One of the best advantages of multithreading is that we can execute different threads at the same time faster and one error does not affect the other threads. Multithreading is very helpful but it also has disadvantages, one example is that the bugs are hard to identify. It can be running with no errors on the first execution of codes but errors may show after the second execution of codes.

2. **Rationalize the use of multithreading by providing at least 3 use-cases.**

   Dimaguiba:

   - A modern computer system is used to run several applications, processes, and accepting and returning outputs. Multithreading allows the computer to manage its resources and distribute the required power and memory in running those tasks. It also allows for multiple users to interact with the system. Using threads to process connections on the line or on air (websites, forums, real-time chats, etc.). One more good use is in banks, thanks to its properties, to manage multiple transactions and requests with little to no delay.

   Urbano:

   - Multithreading can be used for web servers where it can be used for sequential blocking of user traffic. It can be also used for parallel data processing where there can be large amounts of datasets and it can be processed by different threads. It is also very helpful as it improves the scalability especially on microprocessors. It improves scalability because it can handle multiple executions at the same time.

3. **Differentiate between parallelism and concurrency.**

Dimaguiba:

- Concurrency and parallelism differ in the way they access data. Concurrency is when two or more threads can simultaneously access the same variable or data. Whilst Parallel does the same, but they instead run on split data or their own memory separate from each other. One is about dealing with multiple things at once, while the other is about doing multiple things at once (Rob Pyke).

Urbano:

- Parallelism is performed by running multiple copies of the same program at the same time. This simply means the codes are broken down into smaller tasks which can be executed simultaneously which can help computing resources efficiently. Concurrency on the other hand refers to processing multiple tasks which causes them to overlap. In concurrency, the tasks may or may not interact with each other. Concurrency is effective for small tasks because it uses a single processor, while parallelism is helpful for processing large amounts of tasks because it utilizes multiple processors at once.

Part B:

CODE

```
DSAHOA13 > main.cpp > ...
1   #include <iostream>
2   #include <vector>
3   #include <thread>
4
5   int globalInt = 67;
6
7   void add(int add){
8       globalInt += add;
9       std::cout<<globalInt<<std::endl;
10  }
11
12  int main(){
13
14  std::thread T1(add, 5);
15  std::thread T2(add, 10);
16  std::thread T3(add, 15);
17
18  std::cout<<globalInt<<std::endl;
19  T1.join();
20  T2.join();
21  T3.join();
22
23   return 0;
24  }
25
26
```

OUTPUT (Varies)

```
 '--pid=Microsoft-MIEngine-Pid-x0issffr.rfl' '--
67
72
97
82
PS C:\Users\shosh\Documents\VSCODE>
```

```
ode.cpptools-1.34.4-win32-x64\debugAdapters\bi
qis2kt.2vl' '--stdout=Microsoft-MIEngine-Out-4
 '--pid=Microsoft-MIEngine-Pid-nm5oqihd.z0e' '
67
7787
72

PS C:\Users\shosh\Documents\VSCODE>
```

Analysis

Dimaguiba:
Because of the properties of threads in that allowing them to all run simultaneously, it halts operations (except the threads) in order to make room until the joined threads finish their actions. They still print each time, the difference is the order the operations apply. Data-racing happens as they all try to access the global integer at the same time, that the output can be different each time, when and which values were added at a certain time may vary.

Urbano:
Three threads add 5, 10, and 15 to the same global variable at the same time, so the order they run in is not fixed. This is why the output is different each run, and sometimes the printed values are even mixed together like 7787. The first value 67 comes from main, which prints before the threads finish adding.

Code

```cpp
#include <iostream>
#include <vector>
#include <thread>

std::vector<int> numbers = {12, 45, 7, 89, 23, 56, 34, 78, 90, 11, 67, 5};
int target = 67;
int foundIndex = -1;

void linearSearch(int start, int end){
    for(int i = start; i < end; i++){
        if(numbers[i] == target){
            foundIndex = i;
            std::cout<<"Found "<<target<<" at index "<<i<<" (searched range "<<start<<"-"<<end-1<<")"<<std::endl;
            return;
        }
    }
    std::cout<<"Not found in range "<<start<<"-"<<end-1<<std::endl;
}

int main(){

    std::thread T1([>>]linearSearch, 0, 4);
    std::thread T2([>>]linearSearch, 4, 8);
    std::thread T3([>>]linearSearch, 8, 12);

    T1.join();
    T2.join();
    T3.join();
```

```
    if(foundIndex == -1)
        std::cout<<"Target not found"<<std::endl;
    else
        std::cout<<"Result: "<<target<<" is at index "<<foundIndex<<std::endl;

    return 0;
}
```

Output

```
Not found in range Not found in range 4-70-3

Found 67 at index 10 (searched range 8-11)
Result: 67 is at index 10

Process finished with exit code 0
```

Code Analysis

Dimaguiba:
Just like before, we use threading. In this example, the array was divided into multiple parts, covering different ranges through multiple threads. A target value is set for all the search algorithms to find. All threads simultaneously (at the same time) search their assigned areas for the target value. Once done, it returns the index.
Urbano:
The array was split into three parts, and one thread searched each part using linear search. Thread T3 found the target 67 at index 10, and the other two threads did not find it in their parts. The final result was correct and for a small array like this, multi-threading is not faster than a normal search. It only helps with very large arrays.

Conclusion/Learning:
Dimaguiba
Threading is very useful when it comes to executing multiple tasks simultaneously. It automatically uses or adds resources that they need to run those extra tasks. It improves performance and efficiency for some tasks (search tasks for example). But not always will we need to use multithreading. Data-racing is an example when different tasks try to access the same value or variable, leading to unpredictable behavior and results.

Urbano:
During this activity, I have learned how to create a multithreaded code and apply concurrency. Supplementary activity has helped me understand the concept of multithreading a lot more. They used the library of threads and applied std::thread which means that the programs are working at the same time. Threads run in an unpredictable order which can cause a race condition. This activity has also shown that a task can be divided among threads to work in parallel which can be used to improve performance on large data.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.